

Lia — Tools We Used

A study reference for the libraries and software behind the project.

Every tool used to build Lia, grouped the way the project itself is laid out. Each entry explains the general idea first — the thing worth actually learning — then how Lia specifically uses it. Skip the parts you already know; this is meant to be looked things up in, not read start to finish.

The brain: local language models

Ollama

A program that runs large language models on your own machine and serves them over a small local web API (HTTP, on `localhost:11434`) — the same shape of interface as a cloud AI provider, except nothing leaves the computer. It handles loading the model into GPU/CPU memory, managing context windows, and streaming tokens back as they generate.

In Lia: every one of her replies is a request to Ollama's `/api/chat` endpoint. `llm.py` is the thin wrapper around that API — nothing in the rest of the project talks to Ollama directly.

llama3.2:3b

The actual language model — a neural network with about 3 billion parameters, published by Meta, small enough to run on a 4GB graphics card. “3B” refers to its size; smaller models are faster and use less memory but are worse at nuance, precision, and following long instructions than bigger ones (70B+, which need serious hardware or a cloud host).

In Lia: this is why some things she does badly — precise text manipulation like Morse code — are a hardware-driven ceiling, not a bug to fix. A bigger model would do better; it wouldn't fit on this machine.

nomic-embed-text

A much smaller neural network that doesn't generate text — it converts a sentence into a list of a few hundred numbers (a *vector*) that captures its meaning. Two sentences with similar meaning produce vectors that point in similar directions, which is what makes semantic search possible: compare vectors instead of exact words.

In Lia: every message you've ever sent is embedded and stored. When you bring up a topic, her past turns are compared by vector similarity (*cosine similarity*, a dot product) to find what's actually relevant — not by keyword matching.

Memory: data and math

NumPy

The foundational Python library for working with arrays of numbers efficiently. Plain Python lists are slow for math — NumPy stores numbers in tight, contiguous memory blocks (like C arrays) and does operations like addition, multiplication, and dot products on entire arrays at once, in compiled code, rather than one Python object at a time. This is the single most widely used library in scientific/data Python — worth learning well beyond this project.

In Lia, two distinct uses:

- **Embeddings.** Each embedding is a NumPy array of ~768 floats. Comparing your message to everything she remembers means computing a cosine similarity — $\text{dot}(a, b) / (\text{norm}(a) * \text{norm}(b))$ — against every stored vector, which NumPy does fast even over thousands of memories.
- **Audio.** A sound clip is just a NumPy array too — one number per sample, 16,000 or 22,050 or 24,000 of them per second depending on the model. Piper's output chunks, silence for the voice detector, and microphone input are all NumPy int16/float32 arrays underneath.

SQLite

A complete relational database that lives in a single file on disk — no server process, no setup, just a file you can copy, back up, or inspect with any SQLite browser. Built into Python's standard library (`sqlite3`), so it needed no separate install. The right choice whenever a project needs structured, queryable storage but doesn't have (or want) a database server running.

In Lia: `lia_memory.db` holds five tables — facts, memories (with embeddings stored as JSON text), diary entries, indexed document passages, and alarms. `db.py` is the only module that touches SQL directly.

requests

The standard, simple way to make HTTP calls from Python — GET and POST requests, JSON bodies, headers, timeouts. Almost every Python project that talks to any web API uses this library; it's worth knowing well as a baseline skill.

In Lia: every network call in the whole project goes through `requests` — talking to Ollama, fetching weather from Open-Meteo, and the optional Groq/OpenRouter lookups.

Speaking

Piper

An open-source neural text-to-speech engine, small enough to run on CPU in real time. It converts text into phonemes (speech sounds) using eSpeak NG under the hood, then a neural vocoder turns those phonemes into an audio waveform. Voices are distributed as `.onnx` files (the trained model) plus a `.onnx.json` config (sample rate, phoneme mapping).

In Lia: the default voice, “amy”, at 22,050Hz. `voice.py`'s `PiperEngine` wraps it, with tunable pacing/volume/expressiveness parameters (`SynthesisConfig`).

ONNX / onnxruntime

ONNX (Open Neural Network Exchange) is a standard file format for trained neural networks, designed so a model trained in one framework (PyTorch, TensorFlow) can run in a lightweight, portable runtime without needing the whole training framework installed. `onnxruntime` is Microsoft's runtime that actually executes those files, on CPU or GPU.

In Lia: both Piper and the voice-activity detector are ONNX models. This also produced the project's single biggest bug: `onnxruntime`'s CPU backend defaults to a spinning thread pool that burns CPU continuously rather than sleeping between calls — fixed with one environment variable, `OMP_WAIT_POLICY=PASSIVE`, after it was found consuming ~290% of a CPU core while completely idle.

pyttsx3

A thin Python wrapper around whatever text-to-speech engine your operating system already has built in — SAPI on Windows, `NSSpeechSynthesizer` on macOS. Lower quality than a neural voice, but it needs no download and never fails to exist.

In Lia: the fallback if Piper can't load for any reason — the built-in Windows voice (“Zira” by default), so she can never end up completely silent.

Listening

faster-whisper

A reimplementation of OpenAI's Whisper speech-recognition model, built on CTranslate2 (below) for significantly faster inference than the original, especially on CPU. Whisper itself is a model trained on a huge amount of multilingual audio, which is why it's robust to accents and background noise without any per-user training.

In Lia: runs the `small.en` size, deliberately on CPU so it never competes with Ollama for the 4GB of GPU memory the language model needs. Verified during development that `small.en` gets names right (“Anaya”) that the smaller `base.en` garbles (“Ania”).

CTranslate2

A C++ inference engine specifically optimised for transformer models (the neural network architecture behind almost every modern language and translation model), with aggressive quantisation support — running a model at lower numeric precision (int8 instead of float32) to trade a small amount of accuracy for a large amount of speed and memory savings.

In Lia: the actual engine underneath faster-whisper. Not called directly anywhere in the project's own code — it's a transitive dependency, but the reason transcription is fast enough to run on CPU at all.

Silero VAD (via pysilero-vad)

VAD stands for Voice Activity Detection: a small, fast model whose only job is deciding whether a short chunk of audio (here, 32 milliseconds) contains speech or silence — much cheaper than running full speech recognition on every chunk just to find out if anyone is talking.

In Lia: this is what makes the open microphone possible at all — it decides when you start and stop talking, so a full recording only gets sent to Whisper once you're actually done speaking, with a 0.4-second pre-roll buffer so the first syllable isn't clipped.

sounddevice

A Python binding for PortAudio, a cross-platform C library for real-time audio input and output. It's the layer that actually talks to your sound card — opening a microphone stream, playing back a buffer of samples, choosing devices.

In Lia: both directions — recording your voice for the VAD/Whisper pipeline, and playing back whatever Piper (or the fallback voice) synthesises.

Running as an app

pystray

A small cross-platform library for putting an icon in the system tray (the icons next to the clock) with a right-click menu — the minimum needed to have a program live in the background without a normal window.

In Lia: the orange dot. Filled when listening, hollow when muted, with a menu for toggling voice/mic, forcing a diary write, opening the log, and quitting cleanly.

Pillow (PIL)

The standard Python library for opening, creating, and manipulating images — resizing, drawing shapes, reading pixel data. The name PIL (Python Imaging Library) predates Pillow, which is the actively maintained fork almost everyone actually installs today.

In Lia: draws the tray icon itself — a small circle, filled or hollow — generated in code rather than loaded from an image file.

winsdk

A Python projection of the Windows Runtime (WinRT) APIs — the modern system-level interface Windows itself uses for things like notifications, sensors, and media session control. “Projection” means the C++/COM interface is exposed as ordinary (if async-heavy) Python objects and methods.

In Lia: System Media Transport Controls — the same system behind hardware media keys and the taskbar's media flyout — used to play/pause/skip whatever's actually playing, and to read its title and artist, without needing to know which app is playing it.

PyInstaller

A tool that bundles a Python program, the interpreter itself, and every library it depends on into a single folder (or executable) that runs on a machine with no Python installed at all. It works by statically analysing imports and copying in everything reachable, plus special “hooks” for packages that need extra data files.

In Lia: produces Lia.exe. The build script stages into a temporary folder and swaps it in only on success, specifically because a half-written build sitting where Windows autostart points caused a real, confusing failure (“Failed to import the embedded python interpreter”) during development.

Documents, the internet, and these PDFs

pypdf

A pure-Python library for reading and manipulating PDF files — extracting text, splitting or merging pages, reading metadata. PDFs don't store text the way a plain text file does (they store drawing instructions for where each character glyph goes), so “extracting text” is actually reconstructing it from that lower-level description, which is why extraction quality varies between PDFs and libraries.

In Lia: reads any PDF dropped into her library folder, page by page, before the text is chunked and embedded for later retrieval.

Groq

A company that hosts open language models (like Meta's Llama family, at sizes far bigger than what fits on a home GPU) on custom chips (LPUs) built specifically for fast inference — its whole selling point is speed, not necessarily the newest or most capable models available.

In Lia: one of two optional providers for “look that up.” Fast, but a plain chat-completion call to it has no more access to today's news than the local model does either — it can only answer from what it learned during training.

OpenRouter

A single API that proxies requests to many different hosted models (OpenAI, Anthropic, Google, Meta, and others) behind one key and one request format, billed per-use. Notably, appending `:online` to a model name makes OpenRouter run an actual web search before answering — genuinely current information, not a guess from training data.

In Lia: the preferred “look that up” provider when both keys are configured, specifically because of that live-search mode — verified with a real, current, cited answer (a 2025 sporting result) neither the local model nor a plain Groq call could have known.

Open-Meteo

A free weather API that needs no signup or key at all — just a latitude/longitude and it returns current conditions and a forecast as JSON.

In Lia: answers “what's the weather” directly from live data, deliberately without routing it through any language model — there's nothing for a model to usefully add to a number that's already correct.

ReportLab

A Python library for generating PDF files programmatically — laying out paragraphs, tables, and page templates in code rather than a WYSIWYG editor. Lower-level than most people expect: you build a list of “flowable” objects (paragraphs, tables, spacers) and it handles pagination.

In Lia: what generated this document, and the three others alongside it — `docs/make_pdfs.py` holds the shared styling, and a small script per document builds the actual content.